

torch.jit.optimize_for_inference#

[https://pytorch.org/docs/stable/generated/torch.jit.optimize_for_inference.h](https://pytorch.org/docs/stable/generated/torch.jit.optimize_for_inference.html)

Description:

Perform a set of optimization passes to optimize a model for the purposes of inference. If the model is not already frozen, `optimize_for_inference` will invoke `torch.jit.freeze` automatically. In addition to generic optimizations that should speed up your model regardless of environment, prepare for inference will also bake in build specific settings such as the presence of CUDNN or MKLDNN, and may in the future make transformations which speed things up on one machine but slow things down on another. Accordingly, serialization is not implemented following invoking `optimize_for_inference` and is not guaranteed. This is still in prototype, and may have the potential to slow down your model. Primary use cases that have been targeted so far have been vision models on cpu and gpu to a lesser extent. Example (optimizing a module with Conv->Batchnorm):

Function Signature

```
torch.jit.optimize_for_inference(mod, other_methods=None)
[source]#
```

Return type

Returns:

ScriptModule

Code Examples

```
import torch

in_channels, out_channels = 3, 32
conv = torch.nn.Conv2d(
    in_channels, out_channels, kernel_size=3, stride=2,
    bias=True
)
bn = torch.nn.BatchNorm2d(out_channels, eps=0.001)
mod = torch.nn.Sequential(conv, bn)
frozen_mod =
torch.jit.optimize_for_inference(torch.jit.script(mod.eval()))
assert "batch_norm" not in str(frozen_mod.graph)
# if built with MKLDNN, convolution will be run with MKLDNN
weights
assert "MKLDNN" in frozen_mod.graph
```

Detailed Documentation

Documentation

Perform a set of optimization passes to optimize a model for the purposes of inference.

If the model is not already frozen, `optimize_for_inference` will invoke `torch.jit.freeze` automatically.

In addition to generic optimizations that should speed up your model regardless of environment, prepare for inference will also bake in build specific settings such as the presence of CUDNN or MKLDNN, and may in the future make transformations which speed things up on one machine but slow things down on another. Accordingly, serialization is not implemented following invoking `optimize_for_inference` and is not guaranteed.

This is still in prototype, and may have the potential to slow down your model. Primary use cases that have been targeted so far have been vision models on cpu and gpu to a lesser extent.

Example (optimizing a module with Conv->Batchnorm):